



ДОКУМЕНТАЦИЯ НА ПРОЕКТ

GIANT

СИСТЕМА ЗА ПОДГОТОВКА, ТРЕНИРАНЕ И УСКОРЕН ИНФЕРЕНС НА ЕЗИКОВИ МОДЕЛИ

Автор Антон Иванов Христов

Ръководител Евгени Драгомиров Димов

София

2026

I. ТЕМА

Темата на проекта е разработване на цялостна система за създаване и използване на езикови модели - от подготовката на данните и обучението до ускорен inference и количествена оценка на производителността. Реализацията обединява стабилна инженерна основа за GPT-style езиков модел (GIANT) и изследователски слой за бърза генерация (TiDAR/Anchor-TiDAR) в единен codebase.

Фокусът е върху decoder-only Transformer модели, KV cache оптимизации и сравнение между класически autoregressive decode и хибриден decode режим. В хибридният режим diffusion компонентът се реализира като итеративно подобряване на латентна репрезентация на бъдещето с последваща AR верификация в един и същ forward pass. Целта е практическа система с възпроизводимо обучение, ускорен inference и измерими метрики за speedup/latency.

Пълната документация и най-новия код на проекта може да откриете на:
debeltoni.github.io/SUPER-GIANT

II. АВТОРИ

Антон Иванов Христов

- ЕГН: 0751296460
- Телефон: 0887776969
- Имейл: anton.i.hristov.2021@elsys-bg.org
- Училище: Технологично училище “Електронни системи” към ТУ - София
- Клас: 12

III. РЪКОВОДИТЕЛ

Евгени Драгомиров Димов

- Телефон: 0886694906
- Имейл: evgeni.dimov@ocado.com
- Длъжност: Софтуерен инженер, Ocado Technologies
- Роля в проекта: научен ръководител

IV. РЕЗЮМЕ

Проектът е структуриран в два слоя с ясни роли.

GIANT е базовата инженерна платформа: data pipeline, decoder-only training loop, checkpoint/resume механизъм, ускорен inference с KV cache и стандартизирани benchmark сценарии. Този слой осигурява стабилен, възпроизводим и практически използваем моделен pipeline.

TiDAR/Anchor-TiDAR е изследователското разширение върху същата инфраструктура по вдъхновение от научния труд на NVIDIA Research - Think in Diffusion, talk in AutoRegression. То въвежда sequence-level hybrid decode (draft + verify в един forward pass), структурирани attention маски и допълнителни loss конфигурации за анализ на speed/quality поведението.

Комбинацията между двата слоя позволява едновременно production-oriented експлоатация и научно-изследователска работа в една и съща codebase, без дублиране на инструменти и без отделни несъвместими среди.

1. Цели

Основната цел е изграждане на работещ и разширяем LLM stack, който да даде измерими резултати в три направления: качество на модела, скорост на inference и възпроизводимост на експериментите.

Подцелите са:

- да се реализира data pipeline за подготовка на големи текстови корпуси (ingest, cleaning, sharding, indexing);
- да се реализира надежден training loop с resume и checkpoint recovery;
- да се изгради inference слой с KV cache optimization и измерване на speed/latency;
- да се валидира Anchor-TiDAR подходът с фокус върху Free Token Slots и speedup;
- да се изведе практическа методология за сравнение на decode режими при различен хардуер и различни model scales.

Кратък анализ на потребностите показва, че в наличните open-source решения често липсва интеграция между training и high-quality benchmarking pipeline в единна среда. Обикновено има добри отделни компоненти, но не и обща, reproducible, end-to-end структура с лесна поддръжка на експериментален слой. Настоящият проект адресира именно тази празнина.

2. Основни етапи в реализирането на проекта

Проектът е реализиран от един автор, поради което всички дейности са изпълнявани последователно в единна инженерна рамка:

1. Дефиниране на архитектурни изисквания и структури на конфигурации.
2. Изграждане на data pipeline и тренировъчни datasets.
3. Реализация на training pipeline за decoder-only Transformer.
4. Реализация на inference pipeline с KV cache optimization.
5. Добавяне на TiDAR/Anchor-TiDAR изследователски модул.
6. Провеждане на benchmark кампания и анализ на резултатите.
7. Документиране, верификация и подготовка на финални артефакти.

Ролята на научния ръководител е консултативна: насочване към по-добри инженерни практики, критерии за валидиране на резултатите и структуриране на експерименталната методология.

3. Ниво на сложност на проекта

Нивото на сложност е високо, защото системата комбинира няколко трудни слоя едновременно:

- **Numerical complexity** - работа с големи тензори, mixed precision режими, JAX/XLA compilation constraints и memory-bound behavior.
- **System complexity** - синхронизация между local CPU workflow и remote GPU execution, reproducible environments чрез Docker, надеждна работа с артефакти.
- **Algorithmic complexity** - едновременно поддържане на класически autoregressive decode и sequence-level hybrid decode с допълнителна masking логика.
- **Evaluation complexity** - коректно измерване на speedup, latency, acceptance и quality indicators без да се смесват compile overhead, warmup ефекти и run-to-run variance.

Проектът решава практически значим проблем: как един и същ модел да се използва едновременно за високо качество и за висок inference throughput, без задължителна зависимост от втори “draft” модел и без нарушаване на устойчивия инженерен цикъл.

4. Логическо и функционално описание на решението

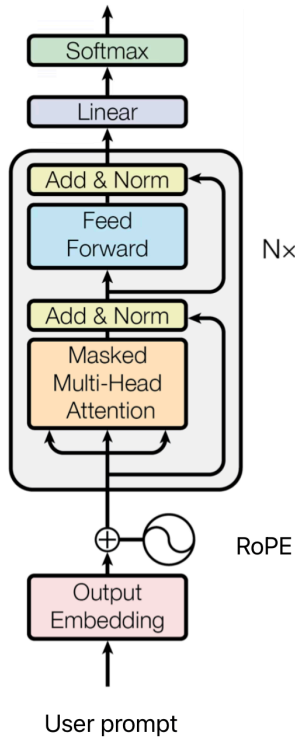
Системата е модулна и е организирана в два главни слоя:

- **GIANT Core** - стабилен training/inference stack.

- **TiDAR Extension** - изследователски слой за hybrid decode и допълнителни loss/mask механизми.

Взаимодействието между модулите е организирано чрез shared конфигурации, стандартизирани entry points и общ artifact storage модел.

4.1. Базова архитектура на модела



Decoder-only архитектурата е организирана като последователност от attention + feed-forward блокове с residual връзки и нормализация между тях.

Входният текст се преобразува в embedding представяния, след което преминава през поредица от causal Transformer блокове. На изхода се получават logits върху речника, които чрез softmax формират вероятности за следващ token.

При training обработката е паралелна в рамките на контекстния прозорец, а при inference се използва prefill + decode режим с KV cache.

Figure 1: Decoder-only Transformer архитектура, използвана като базов модел

Базовият модел е decoder-only Transformer с causal masking. Всеки training sample се обработва като последователност от токени, а целевата функция е next-token prediction. Архитектурно са използвани съвременни компоненти като RMSNorm, RoPE и SwiGLU, което осигурява стабилност при обучение и добра ефективност при inference.

Multi-head attention операторът е:

$$h_h = \text{softmax} \left(\frac{(QW_h^Q)(KW_h^K)^T}{\sqrt{d_k}} + M \right) (VW_h^V)$$

$$\text{MHA}(Q, K, V) = [h_1 \parallel h_2 \parallel \dots \parallel h_H] W^O$$

където M е причинно-следствена маска, която гарантира, че позиция i вижда само токени с индекс $j \leq i$.

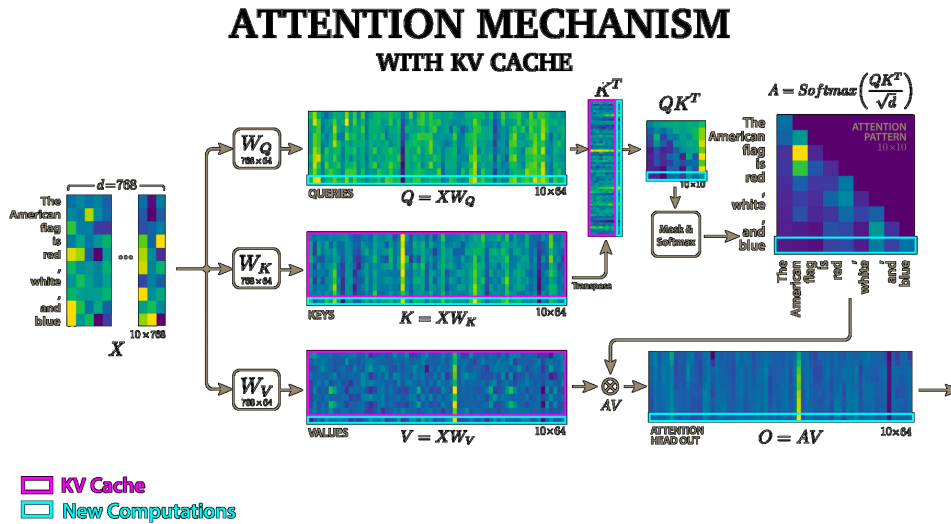


Figure 2: Attention механизъм и контекстно претегляне

4.2. Функционални модули и зависимости

Функционално решението е изградено от следните модули:

- **Data Module** - ingest, cleaning, dataset sharding, статистики, проверка на входните файлове.
- **Training Module** - optimizer loop, stages, gradient accumulation, checkpoint/recover.
- **Inference Module** - prefill/decode, sampling режими, KV cache, performance counters.
- **Evaluation Module** - benchmark сценарии и сравнение на ключови метрики.
- **Deployment Module** - Docker runtime, remote execution, data sync, artifact management.

Комуникацията между модулите е конфигурационно-ориентирана: модулите не разчитат на ръчно редактирани вътрешни параметри, а на explicit YAML settings, което намалява риска от скрити зависимости.

4.3. Data и training pipeline

Data pipeline изпълнява нормализация, филтриране, токенизация и shard-ване, а получените shard файлове се подават директно към training loop-а за предвидим I/O.

Training pipeline е stage-based: всеки stage има собствени параметри (контекст, learning schedule, loss weights), а състоянието се пази с checkpoint, което позволява надежден resume при прекъсване.

Базовите training термини се дефинират така:

$$L_A = -\frac{1}{N} \sum_{t=1}^N \ln p(y_t | y_{<t}), \quad L_D = -\frac{1}{N} \sum_{t=1}^N \ln p_{\theta}(y_t | c_t)$$

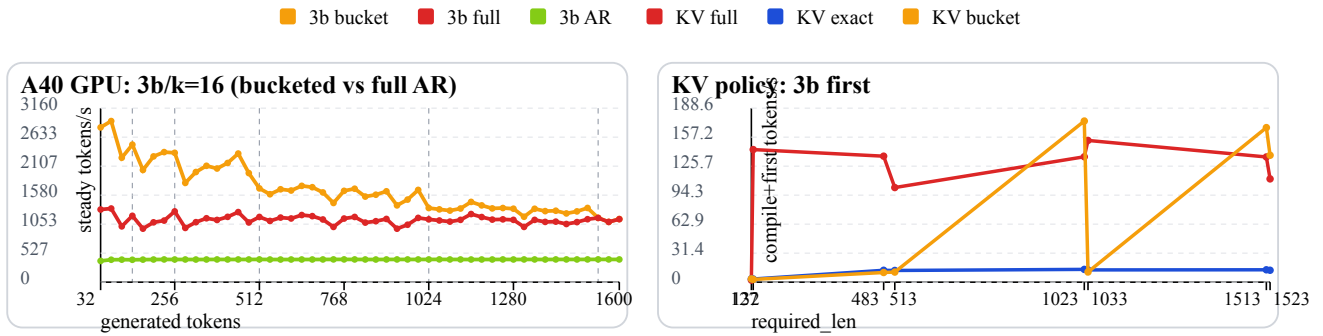
където L_A е autoregressive cross-entropy термин, L_D е diffusion/draft термин, N е броят обучителни позиции в batch-a, y_t е таргет токенът на позиция t , а c_t е контекстът, достъпен за diffusion клона при съответната маска.

Общата loss комбинация в Anchor-TiDAR режима е:

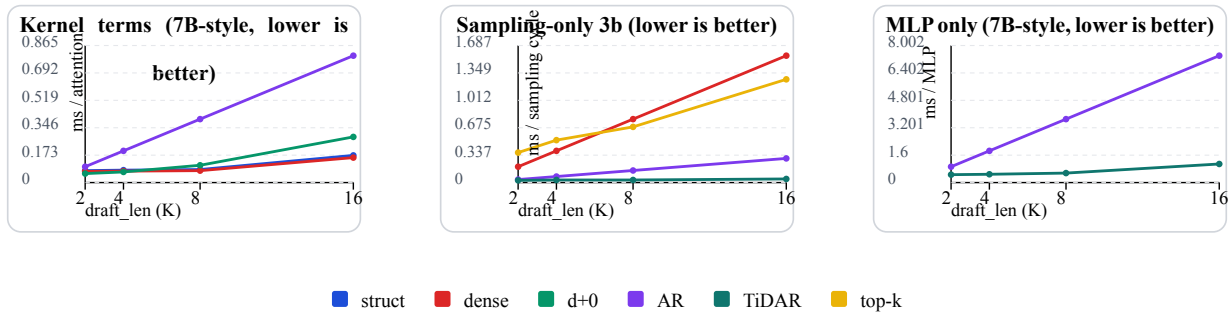
$$L = \alpha L_A + \beta L_D + \rho D_f + \chi D_r + \delta L_H + \delta_m L_P + \eta L_S + \gamma L_K$$

В практическата постановка се използват няколко вида loss термини: (1) базови езикови термини за AR и Diff клоновете, (2) дистрибуционни термини за подравняване между двата клона, (3) маскирани/префиксни термини за стабилност на acceptance веригата и (4) допълнителни регуляризационни термини за контрол на training динамиката.

$$D_f = \sum_v p_A(v) \ln \frac{p_A(v)}{p_D(v)}, \quad D_r = \sum_v p_D(v) \ln \frac{p_D(v)}{p_A(v)}, \quad L_H = - \sum_t m_t \ln p_D(y_t | c_t)$$



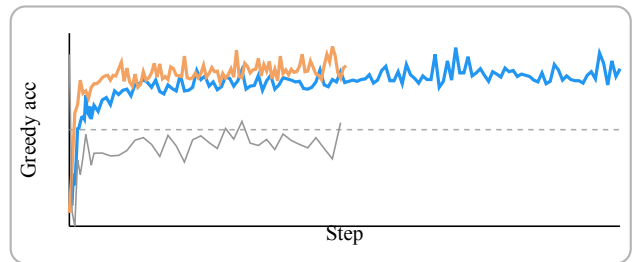
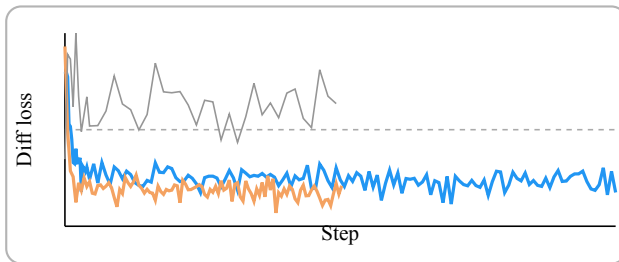
Двете графики показват как политиката за оразмеряване на KV cache влияе директно върху decode производителността: bucketed подходът запазва по-висока steady скорост спрямо full-context baseline, а в first-run режима се вижда цената на compile и ефектът от cache sizing върху първата заявка.



Тези 3 графики показват Free Token Slots феномена при attention, sampling и MLP.

SmolLM 135M vs 360M

135M 360M ratio 135/360



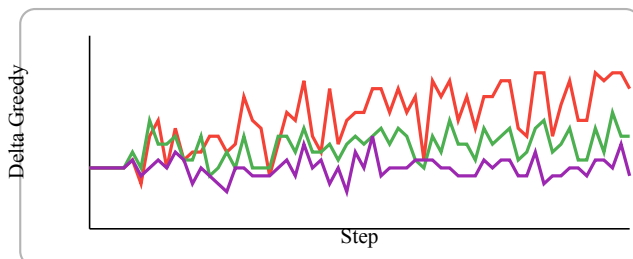
По-големият 360M модел поддържа по-стабилен quality профил при сравним decode режим, което е очакван индикатор за по-висок моделен капацитет.

В practical inference план това означава, че 360M дава по-предсказуемо поведение при сходни decode настройки, докато 135M остава полезен за по-евтини и бързи итерации. Тази двойка графики е полезна за избор на model size спрямо budget/quality изисквания.

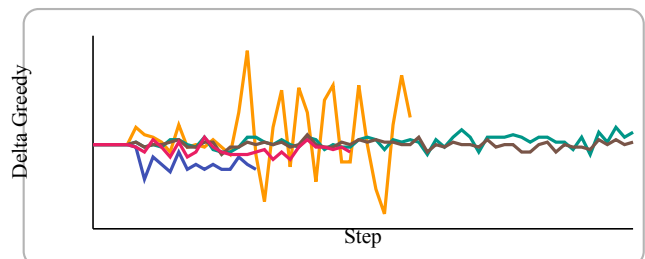
Stable baseline KL-only no-diff KL-keep soft-align Distill AR->Diff
G-eta hard-agree B-beta diff-boost Top-K set-match B-gamma hard+set D-mask prefix-only

Следващите графики показват delta спрямо stable baseline за core и за разширения набор от loss конфигурации върху TinyStories (90k-121k).

Core loss варианти



Разширен loss набор



Анализ: TinyStories е нискоентропиен корпус с ограничена лексикална вариативност, затова разликите между loss вариантите често са с малка амплитуда. За по-ясно разделяне на ефектите е необходимо скалиране към по-големи модели и по-трудни корпуси.

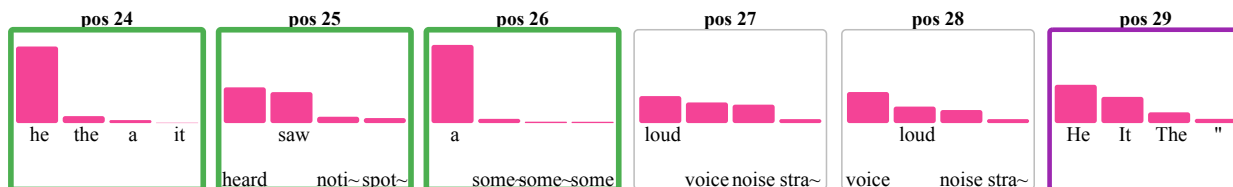
Core вариантите са добър избор за стабилност и контрол, докато разширеният набор е по-подходящ за търсене на агресивни speedup хипотези.

Логитни хистограми (TinyStories примери)

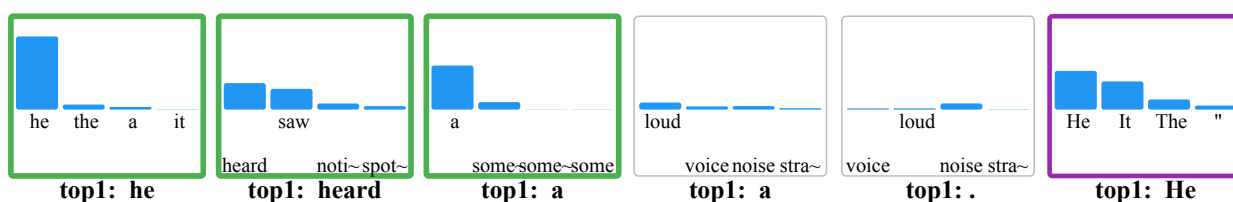
За всяка позиция са показани два панела: горният е AR verify разпределението, а долният е Diff draft разпределението за същите токени в същия ред.

Prompt: “In the forest, a tiny fox”

AR verify logits (top-4, sorted by AR)



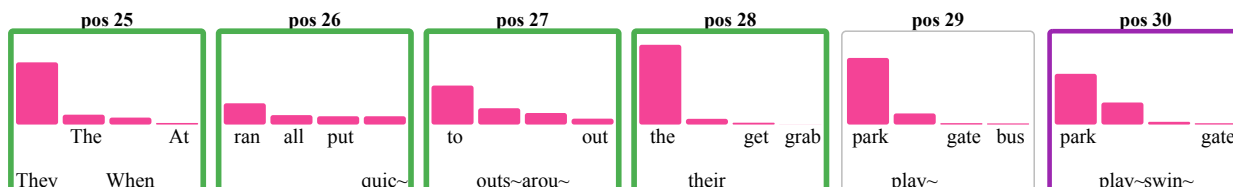
Diff draft logits (same tokens, AR order)



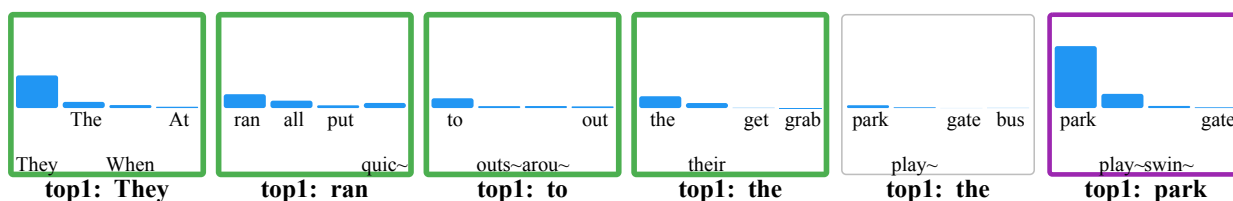
Sentence context Suddenly, he heard a loud noise.

Prompt: “One day, the teacher said”

AR verify logits (top-4, sorted by AR)



Diff draft logits (same tokens, AR order)



Sentence context They ran to the park and saw a big slide.

След първия reject често се наблюдават локални top-1 съвпадения между AR и Diff, но acceptance веригата остава прекъсната за текущата итерация, поради което позициите не се маркират като приети.

4.4. Anchor-TiDAR: hybrid decode лoгика

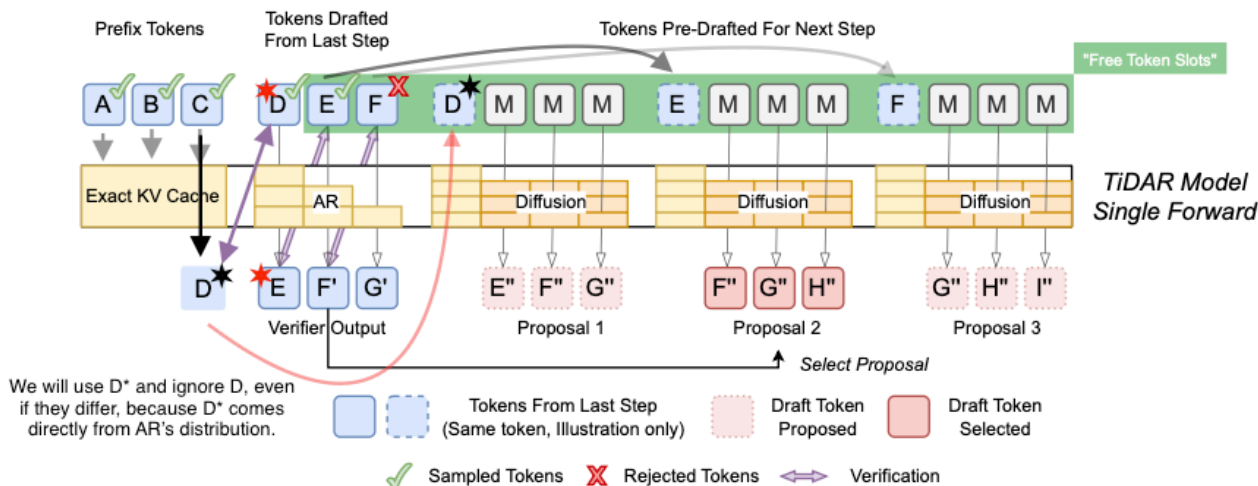


Figure 3: Anchor-TiDAR forward pass: verify + predraft в един моделен проход

Anchor-TiDAR въвежда sequence-level hybrid decode: системата генерира draft предложения паралелно и ги верифицира autoregressively в рамките на един forward pass със специализирани маски. Това позволява по-добро използване на GPU паралелизма спрямо класическия AR decode път.

Логически decode цикълът е:

1. prefill на контекста и инициализация на KV cache;
2. генериране на draft кандидати;
3. verify стъпка и приемане/отхвърляне на префикс;
4. commit/rollback на pointer-и в KV cache;
5. преход към следваща итерация.

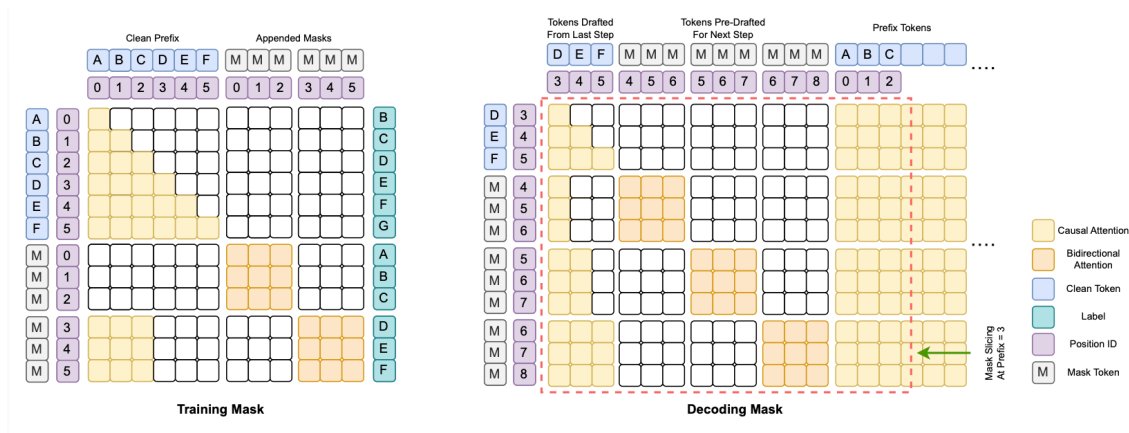


Figure 4: TiDAR маски: decode и training с обща легенда

Практическата полза е, че `verify/predraft` изчисленията се амортизират в един компактен `execution path`, което намалява относителния `overhead` при `decode`.

4.5. Free Token Slots и GPU ефективност

Концепцията Free Token Slots описва случаи, при които в дадена decode итерация има неизползван паралелен ресурс, който може да бъде запълнен с полезна допълнителна работа. Този анализ е ключов за latency-critical deployment сценарии, защото показва къде архитектурните промени носят реална speedup стойност.

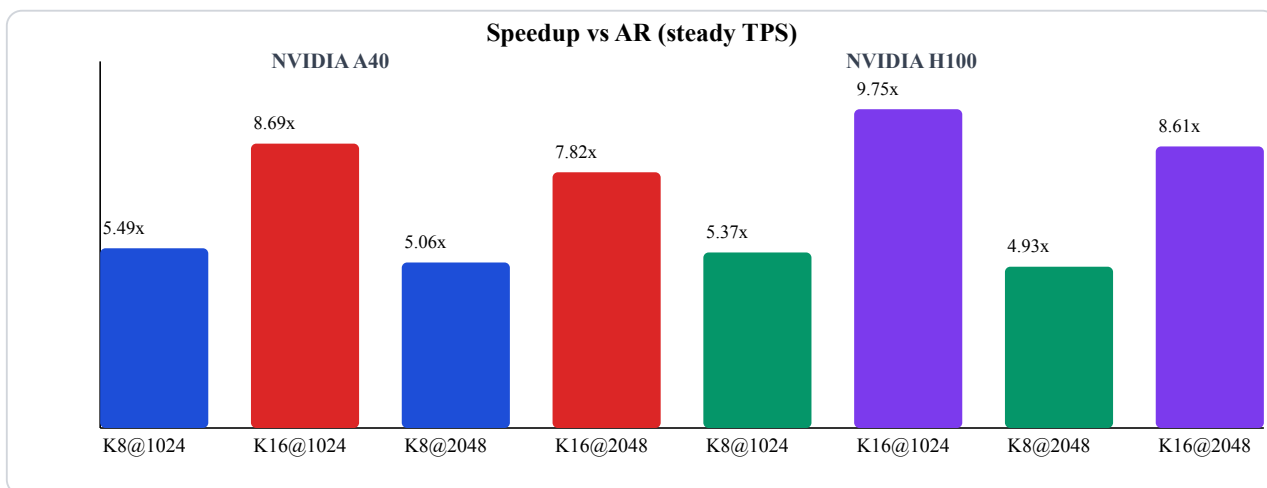
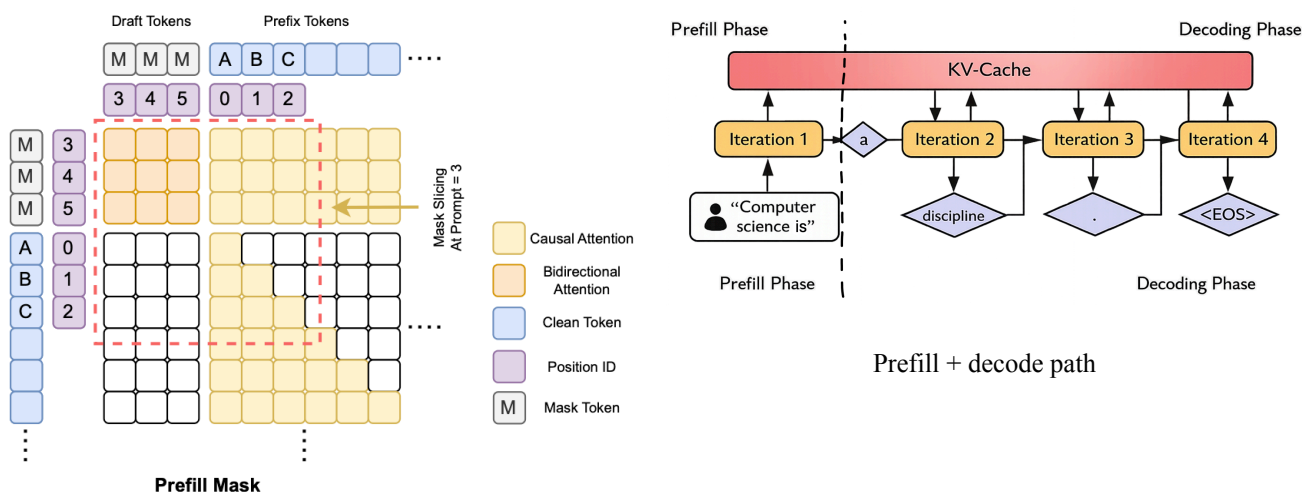


Figure 5: Head-to-head speedup диаграма (steady TPS), рендерирана директно от Typst

Валидираните резултати в този проект показват, че hybrid decode подходът е устойчив при мащаби до 7B параметри и може да осигури значимо ускорение спрямо AR baseline при запазване на конкурентно качество.

4.6. End-to-end decode път (prefill + decode)



TiDAR prefill mask

Figure 6: End-to-end decode път в GIANT/TiDAR: prefill маска (ляво) и decode pipeline (дясно)
Практически inference изпълнението е разделено на две фази. В **prefill** фазата целият prompt се обработва наведнъж, като се инициализира KV cache състоянието за последващото генериране.

В **decode** фазата системата добавя нови токени итеративно, като използва кешираната история вместо повторно пресмятане на целия контекст.

При GIANT този процес следва класически AR модел. При TiDAR/Anchor-TiDAR към същия execution skeleton се добавят verify/predraft стъпки и структурирани маски, без да се нарушава базовата консистентност на KV cache. Това позволява сравнение между режимите в една и съща инфраструктура и при едни и същи входни условия.

Поради тази унифицирана структура измерванията за скорост, закъснение и стабилност са директно съпоставими между AR и hybrid decode вариантите, което е критично за обективна оценка на реалния practically usable speedup.

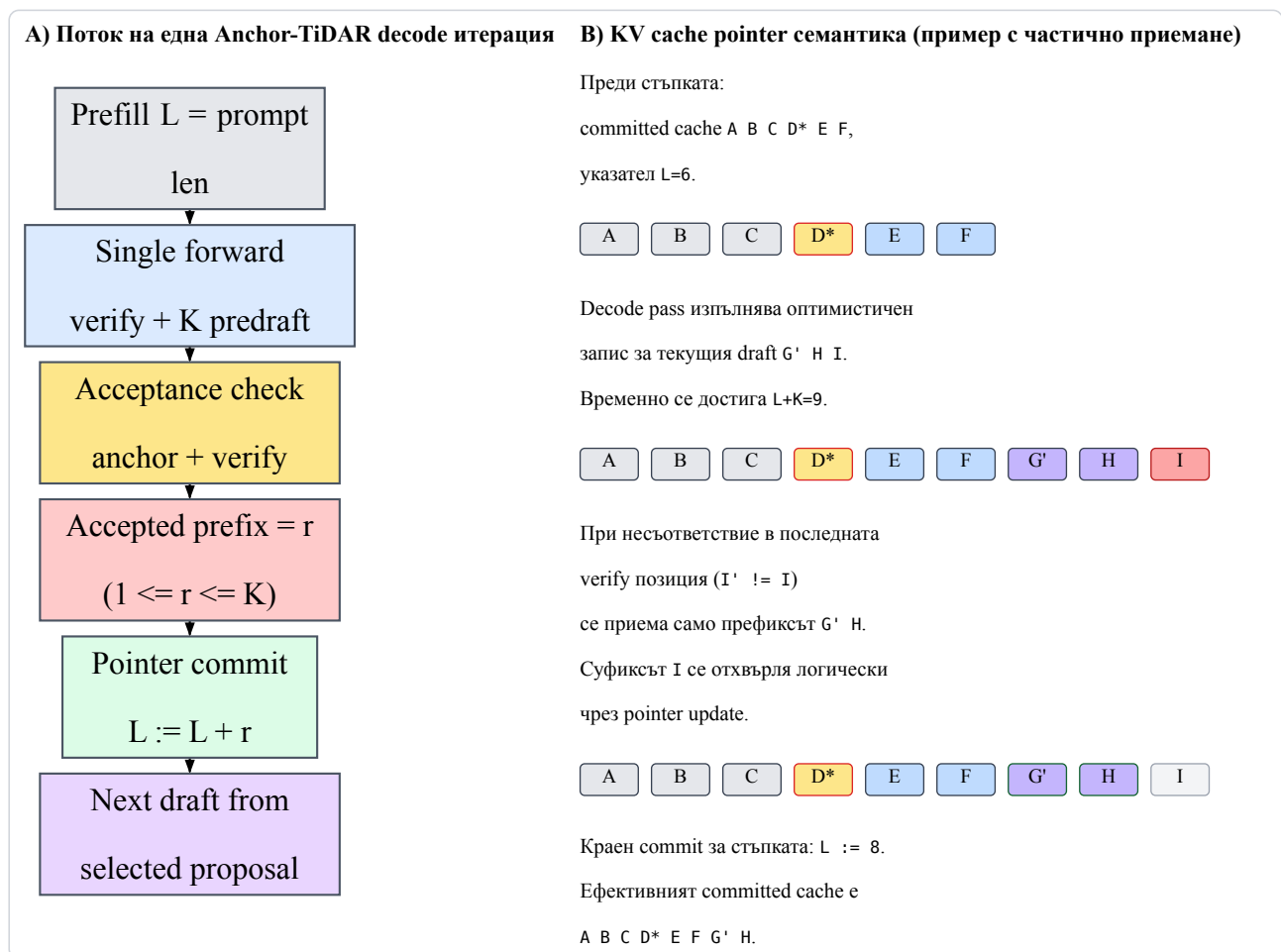


Figure 7: KV cache commit/rollback при Anchor-TiDAR: поток и pointer update

5. Реализация

Изборът на технологични средства е направен с фокус върху надеждност и ефективност:

- **Python** като основен език за бърза разработка и богатата ML екосистема.
- **JAX/Flax/Optax** за висока числена производителност, модулност и устойчив checkpointing.
- **Docker + S3 workflow** за reproducible runtime, пренос и съхранение на големи данни.
- **Remote GPU execution** за тежки training/inference run-ове.

JAX стекът е особено силен в този проект, защото комбинира **JIT** компилация и **XLA** оптимизации с функционален модел на изчисление. На практика това позволява training/inference функциите да се компилират до оптимизиран изпълним граф за конкретни tensor shapes и така да се намали host overhead. Flax дава ясен модел за архитектура и параметри, Optax осигурява стабилна optimizer/scheduler логика, а Orbx гарантира надеждно checkpoint възстановяване при дълги run-ове.

Използваните алгоритми включват:

- decoder-only autoregressive training;
- speculative/hybrid decode стратегии;
- KV cache bucket policy;
- анализ на acceptance и локални logits профили;
- loss composition за TiDAR post-training sweep.

От инженерна гледна точка изборът на този стек е обоснован и с изискването за дългосрочна поддръжка. Конфигурациите, checkpoint файловете и benchmark артефактите са съвместими между локална и remote среда, което позволява един и същ експеримент да бъде повторен при идентични параметри и сравним хардуер.

Допълнително, modular entry-point структурата намалява риска от монолитни скриптове с неявни зависимости. Всеки сценарий (data, train, inference, evaluate) е отделен и може да бъде автоматизиран в CI/automation pipeline.

Използваната литература включва фундаменталните Transformer публикации, работи за serving оптимизации и референтната статия за TiDAR.

6. Описание на приложението

6.1. Стартиране и инсталация

Препоръчителният начин за стартиране е чрез предоставения Docker контейнер (наличен автоматично на [docker.io/bonanc/giant-training:latest](https://hub.docker.com/r/bonanc/giant-training) или построен локално от CICD папката), но приложението може да се изпълнява и директно в Linux/macOS/WSL2 среда с Python. Типичният процес е:

1. клониране на репозиторието;
2. настройка на dependencies/venv или Docker контейнер;
3. проверка на конфигурационните файлове;
4. стартиране на data pipeline, training или inference entry point.

Примерни команди:

```
python GIANT/v2/data_pipeline/build_corpus.py --config OpenWebText_1b.yml
```

```
python GIANT/v2/model/Run_training.py --resume latest --batch 64
```

```
python GIANT/v2/model/Generate_faster.py --prompt "Steve Jobs is"
```

```
python TiDAR/model/Run_training.py --config TiDAR/model/Config_135m.yml
```

```
python TiDAR/model/inference.py --prompt "What is KV cache" --draft_len 8
```

6.2. Използване

Потребителят работи изцяло през CLI, като управлява run параметрите чрез YAML конфигурации и command flags. Основните режими са:

- подготовка на данни;
- обучение (базов режим или TiDAR режим);
- inference (AR или Anchor-TiDAR);
- benchmarking и анализ на метрики чрез Typst.

6.3. Поддръжка

Поддръжката е организирана около три практики: версия на конфигурациите, периодично архивиране на checkpoints/логове и проверка за съвместимост между model state и tokenizer. При промени в кода се изпълнява кратък benchmark run за регресии, за да се запази проследимостта на резултатите.

7. Заключение

Проектът GIANT + Anchor-TiDAR изпълнява основната си цел: реализирана е цялостна и работеща платформа за езикови модели, която покрива всички ключови етапи - подготовка на данни, обучение, инференс и измерване на производителността. Всичко това е обединено в една обща codebase с възпроизводим процес на изпълнение. Системата може да се използва както за практически deployment сценарии, така и за контролирани изследователски експерименти. Ключовият принос е архитектурното разделяне между стабилен core слой (GIANT) и изследователски extension слой (TiDAR). Това позволява паралелно развитие на reliable training/inference pipeline и на нови decode идеи, без нарушаване на базовата експлоатация. В operational план платформата поддържа checkpoint/resume, ясни CLI entry points, KV cache оптимизации и проследими експериментални артефакти.

Емпиричните резултати показват, че hybrid decode подходът (draft + autoregressive verify в един forward pass) може да съчетае висока скорост и запазено качество. В проведените head-to-head измервания върху модели до 7B параметри е отчетено ускорение от 4.93x до 9.75x спрямо AR baseline при стабилно inference поведение в различни GPU режими. Ограничението в текущата версия остава loss sweep експериментът, който трябва да се разшири към по-голям model scale, за да се разграничат по-ясно фините ефекти между отделните loss комбинации.

В близкото бъдеще развитието на проекта е насочено към:

- multi-GPU distributed training и по-големи тренировъчни диапазони;
- разширяване и скалиране на базовия модел към диапазон 300M-1B параметъра;
- обучение върху по-богати и по-разнообразни корпуси (OpenWebText, Wikipedia, OpenCrawl и сходни източници) за по-добра обобщаемост;
- доразвитие на serving слоя за по-висок паралелизъм на заявки и по-ефективно KV cache управление;
- експерименти с архитектурни вариации за decode speedup (MoE, MLA и сходни подходи) и проследяване на ефекта им върху Free Token Slots.